

CYBER PHYSICAL SYSTEMS PROJECT
MODEL PREDICTIVE CONTROL - GYM CAR
RACING

Alessandro Cesa

July 2024

Contents

Introduction	2
Problem Statement	2
System Requirements	3
Model Predictive Control	3
Results	4
Training	4
Control	5
Conclusions	6

Introduction

This project is on a Model Predictive Control problem, applied to the Open-AI Gym Environment "Car Racing".

The task consists in building a control policy for a simulated self driving car, that needs to be driven along a street track. We will define our requirements using Linear Temporal Logic, then we'll simulate some random episodes to create a dataset on which we'll train a Deep Neural Network to predict the system. At this point we use the Neural Network to predict the system and control the car with model predictive control.

All the code used for the project is available at <https://github.com/AlessandroCesaTs/Car-Racing>

Problem Statement

"Car Racing" is an environment from Open-Ai's gym project. The environment consists a colored image of a car on a street track surrounded by grass; but we only see the 96x96 portion surrounding the car. We can perform 4 different actions: doing nothing, steering left, steering right, accelerating or braking. The goal is to drive the car around the track without making it go out.



System Requirements

Our car always needs to be moving, otherwise a simple reward hack to always stay on track would be to never move, since the episode begins with the car on the track. We want the car to always be on track or, at least, if it goes out of track we want it to go back. Also, we always want the track to be in the portion of environment that we're able to see, otherwise it would be impossible for us to go back on track.

We can describe our requirements for the system, using linear temporal logic:

If we define $x(t)$ = position of car at time t , y = the car is on track, z = The grass is in the visible portion of the environment

Hard Requirements:

- Requirement H1: The car always needs to be moving: $G(x(t) \neq x(t+1))$
- Requirement H2: We always need to see the track : $G(z)$
- Requirement H3: Eventually we always need to go back to the track: $G(F(y))$

Soft Requirements:

- Requirement S: We always have to be on track: $G(y)$

Model Predictive Control

We'll first do some pre-processing on the environment to make it simpler: we turn the image in grayscale, and we use only 4 colors: black for the car, a darker gray for the street, and 2 lighter greys for the grass.

Then, we will consider as our state, transitions of 4 consecutive frames, in order to be able to get some insight about speed and direction.

Following the framework used by Kapoor, Parv, Anand Balakrishnan, and Jyotirmoy V. Deshmukh. in "Model-based reinforcement learning from signal temporal logic specifications." (2020), we first simulate 50000 random episodes, where we force the car to accelerate and to do consecutive right turns and consecutive left turns, in order to explore the whole environment. This is our dataset

Then, we train on this dataset an auto-encoder neural network to predict the system: given the last 4 frames and the action that we perform, the model will generate a prediction of the next frame.

Now that we have our model of the system, we can actually perform the model predictive control. At each timestep, we want to choose the best action to satisfy the robustness of our requirements on a certain time horizon τ .

We will force to take the action "gas" at least once every 4 time-steps, in order to have the car always moving. As robustness indicators for the other requirements we will use two functions: the distance from the street and the distance from the grass. We would like the distance from the street to be always 0, so we'll have the car on the street, so we want to minimize it. Instead we

want to maximize the distance from the grass, because this will mean to be at the center of the street . So we'll try to maximize distance from grass-distance from street.

Chosen a time horizon, this is the Model Predictive control algorithm

Algorithm 1 Model Predictive Control using F system predictor

```

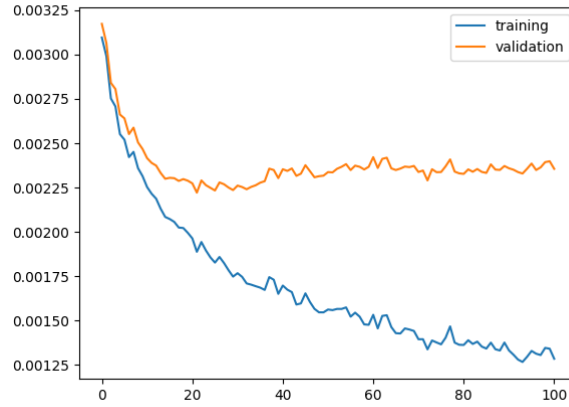
for Each time step  $t$  in episode do
  for  $a_t$  in possible actions do
    for  $i \in \{0, \dots, \tau\}$  do
       $\hat{s}_{t+i+1} = F(\hat{s}_{t+i}, \hat{a}_{t+i})$ , choose  $a_{t+i+1}$  as the one that minimizes the
      cost  $\rho(\hat{s}_{t+i+1})$ 
    end for
  end for
  Execute first action  $a^*$  from the sequence  $(a_t, \dots, a_{t+\tau})$  that minimizes
  the total cost
end for

```

Results

Training

We found the best learning rate to be 0.001; we can see that the model reaches its best at epoch 21 and then it starts to overfit.



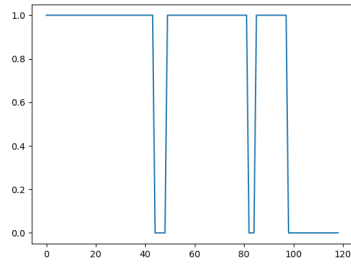
Loss during training

Control

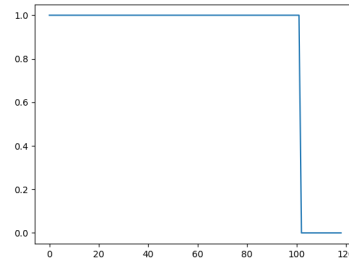
We tried different time horizons, we can analyze two of them; the videos are found in the github repository.

We have the plots of two robustness indicators: distance from street and distance from grass, as well as the plots from the boolean signals that tell us if the car is on the street and if the street track is inside the viewable space

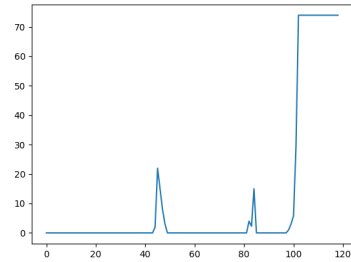
Time-Horizon=15



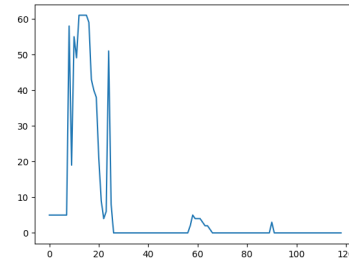
Car is on street



Street is viewable



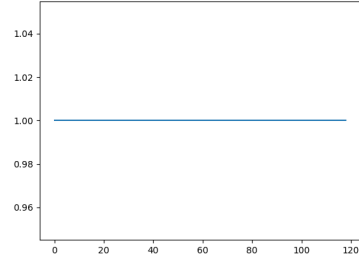
Distance from street



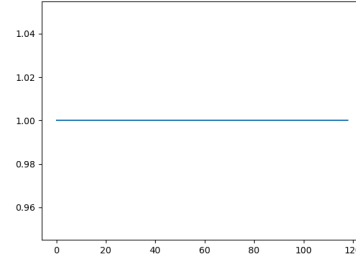
Distance from grass

We can see, both from the plots and from the video, that the none of the requirements, a part from H1 which is enforced is actually satisfied; We get put of track, so the requirement S is violated, at the end we don't see the street anymore, so H2 gets violated, and while in the first part of the episode we go back in track when we get out, at the end we never go back, so in the last part also H3 gets violated.

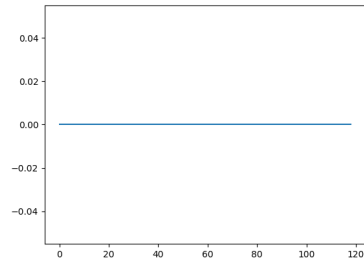
Time-Horizon=20



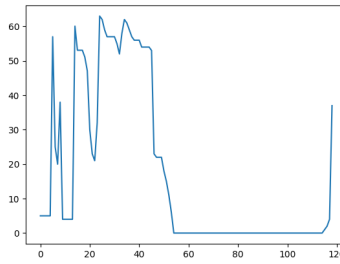
Car is on street



Street is viewable



Distance from street



Distance from grass

Here instead it looks like that we're able to satisfy all of the requirements; however, we can see that from a certain point, both the distance from the street and from the grass are 0: this means that the car is half on street and half on grass, as we can also verify from the video. So in this episode the requirement S is not completely satisfied, since we're not completely on track. So we can for sure say that the time horizon of 15 frames is the best one

Conclusions

We were able to build a control system which, even if not completely bad, still needs a lot of improvement. Even though we managed to get an episode where all the hard requirements were satisfied and the soft one was almost satisfied, the control of the car is not perfect and smooth at the center of the road. Also, not all of the episodes were the same, there was some random behavior, so we had some lucky episodes and some unlucky ones.

A possible explanation is in the poor training of the predictive model.

A possible improvement would be that of using a Variational Auto Encoder instead of an Auto Encoder for predicting the next frame, since VAEs are usually more suitable for generative tasks